

Lab Assignment

International Cybersecurity and Digital Forensics Academy

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

1. Purpose of the Lab Assignment

The purpose of this lab is to understand how steganography can be used to hide information inside an image file. The practical work involves preparing and preserving an original carrier image, creating a secret evidence file, embedding the file using Steghide, and extracting it with the correct password. The lab also teaches how to use SHA-256 hashes to verify that evidence has not been changed. In addition, StegSeek is used to perform a controlled password dictionary test. The exercise helps develop basic digital forensic skills for identifying, recovering, and documenting hidden data.

2. Lab Report Structure

The report explains how the carrier image was prepared, preserved, and verified using its file type and SHA-256 hash. It documents the creation of the secret evidence file and the process of hiding it inside a new stego image using Steghide. The extraction process is recorded and the original and recovered files are compared using hashes and file comparison tools. The report also compares the original and stego images by checking their sizes, hashes, initial bytes, and readable strings.

2.1. Title Page

- Lab Title: Lab 4 — Steganography and Hidden Data Detection
- Student Name: Olanrewaju Wale Mustapha Oladimeji.
- Student ID: 2025/FWSD/11320.
- Course Name: SBT-DF202 — Computer and Digital Forensics
- Instructor Name: Aminu Idris
- Date of Submission: 4/09/2026
- Version:

2.2. Executive Summary

This lab focused on the use of steganography to hide and recover information from an image. A carrier BMP image was preserved, hashed, and used to create a new stego image containing a controlled secret evidence file. Steghide was used to embed and extract the hidden file with the required password, while SHA-256 hashes were used to confirm the integrity of the recovered evidence. A small wordlist was also tested with StegSeek to demonstrate a controlled password-recovery workflow. The practical will improved student understanding of hidden data analysis and basic digital forensic evidence handling

2.3. Lab Objectives

- The lab objective explain the difference between steganography and encryption
- It teaches students how to understand insertion, substitution, and least significant bit (LSB) techniques.
- How to use Steghide to embed and extract hidden information.
- Students also learn to calculate and compare SHA-256 hashes to verify file integrity.
- How to compare the original and stego images using file size, bytes, and strings.

2.4. Tools and Resources Used

The following tools and resources were used during the lab:

- Kali Linux: Operating system used for the practical exercise.
- Steghide: Used to embed and extract hidden files from the image.
- Stegcracker: Used for the controlled password dictionary test.
- SHA-256 / sha256sum: Used to calculate and compare file hashes.
- Xxd: Used to examine the initial bytes of the image files.
- Strings: Used to identify readable text within the files.
- Diff: Used to compare the original and extracted secret files.
- Nano: Used to create and edit the secret evidence and wordlist files.
- BMP Carrier Image: Authorised lab image used as the carrier file.
- Wordlist: Small controlled wordlist containing the test password.

```
(water@kali)~[~/Desktop]
$ cd steg

(water@kali)~[~/Desktop/steg]
$ ls
_tower_original_image_for_lab.bmp

(water@kali)~[~/Desktop/steg]
$ md5sum _tower_original_image_for_lab.bmp
7f77e02253d5ae6426c4d0c932325ae3 _tower_original_image_for_lab.bmp

(water@kali)~[~/Desktop/steg]
$
```

Part B — Create the Secret Evidence File

Create a text file named:

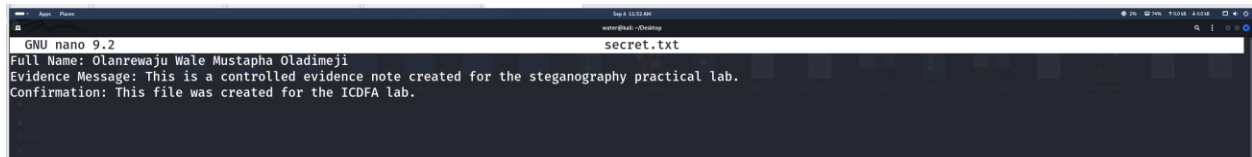
secret.txt

Include:

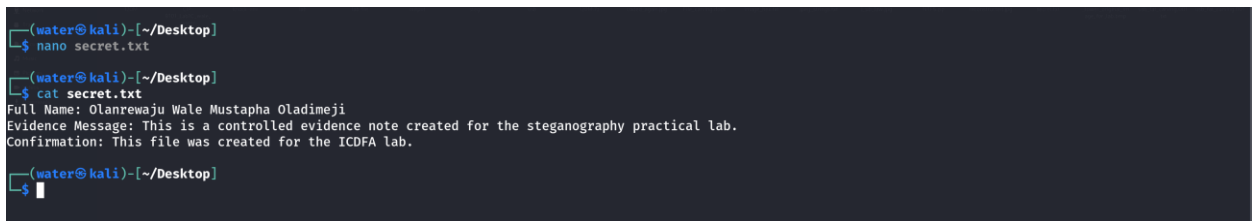
- Your full name
- A short controlled evidence message
- Confirmation that the file was created for the ICDFA lab

Calculate and record the SHA-256 hash of the file.

Creating a file for text secret.txt



```
GNU nano 9.2 secret.txt
Full Name: Olanrewaju Wale Mustapha Oladimeji
Evidence Message: This is a controlled evidence note created for the steganography practical lab.
Confirmation: This file was created for the ICDFA lab.
```



```
(water@kali)-[~/Desktop]
$ nano secret.txt

(water@kali)-[~/Desktop]
$ cat secret.txt
Full Name: Olanrewaju Wale Mustapha Oladimeji
Evidence Message: This is a controlled evidence note created for the steganography practical lab.
Confirmation: This file was created for the ICDFA lab.

(water@kali)-[~/Desktop]
$
```

Calculate and record the SHA-256 hash of the file.



```
(water@kali)-[~/Desktop]
$ sha256sum secret.txt
d6ade379de876a3bcc183e5e1798b73f176272733c659e293d38f2ef4cd2b889 secret.txt

(water@kali)-[~/Desktop]
$
```

Part C — Embed the Hidden File with Steghide

Use Steghide to embed secret.txt into a **new stego image**.

For the guided portion of the lab, use the lab password:

1234

The source lab uses this password specifically so the later dictionary-recovery task can be completed.

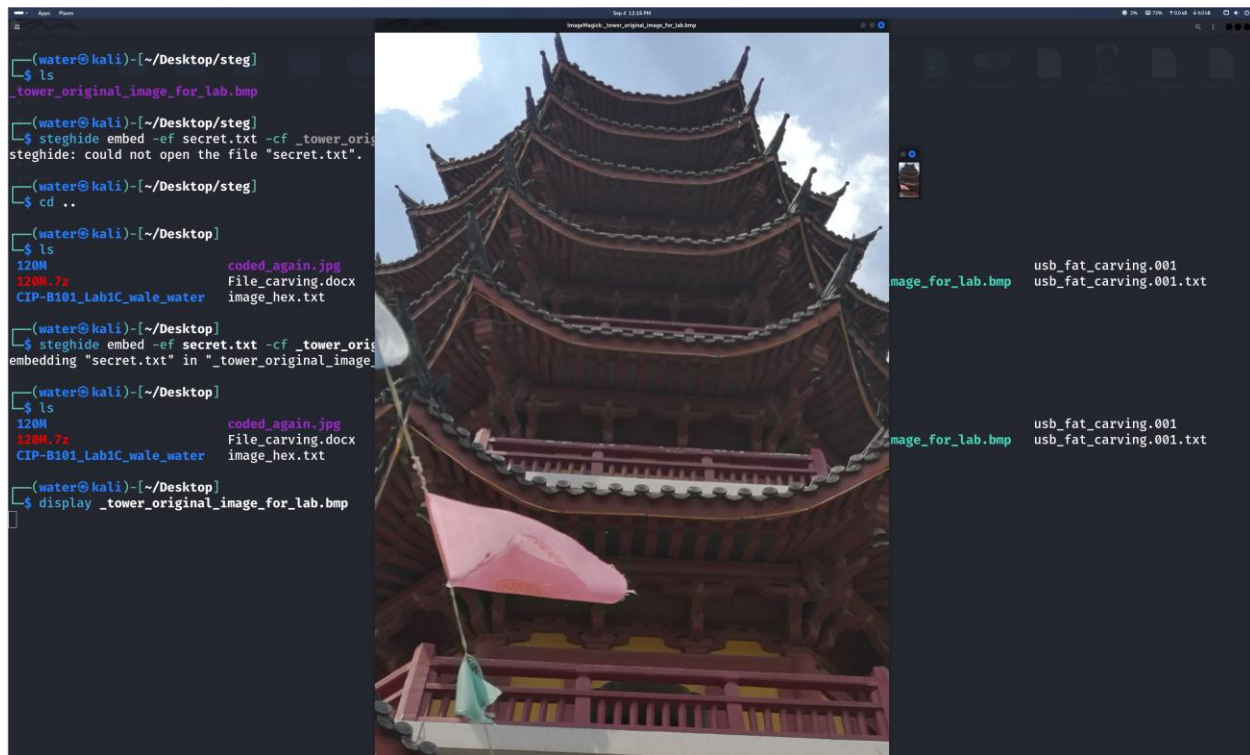
Example structure:

```
steghide embed \ -ef secret.txt \ -cf tower_original_image_for_lab.bmp \ -  
sf tower_stego_lab4.bmp \ -p 1234
```

Calculate and record the hash of the resulting stego image.

Steghide to embed secret.txt into a **new stego image**

```
(water@kali)-[~/Desktop]  
$ ls  
120M      coded_again.jpg      jpeg_strings.txt      lab      output      steg      usb_fat_carving.001  
120M.7z   File_carving.docx    J_ub_law.jpg          'lab file' reconstructed.jpg _tower_original_image_for_lab.bmp  usb_fat_carving.001.txt  
CIP-B101_Lab1C_wale_water image_hex.txt        knock-env             NTFS.zip  secret.txt  usb  
  
(water@kali)-[~/Desktop]  
$ steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -p 1234  
embedding "secret.txt" in "_tower_original_image_for_lab.bmp"... done  
  
(water@kali)-[~/Desktop]  
$
```



hash result stego image is 5a775dba746117c61d8cfc4920a4d3
_tower_original_image_for_lab.bmp

```
(water@kali)-[~/Desktop]  
$ md5sum _tower_original_image_for_lab.bmp  
5a775dba746117c61d8cfc4920a4d3 _tower_original_image_for_lab.bmp  
  
(water@kali)-[~/Desktop]  
$
```

7f77e02253d5ae6426c4d0c932325ae3 _tower_original_image_for_lab.bmp

5a775dba746117c61d8cfc4dc4920a4d3 _tower_original_image_for_lab.bmp

Part D — Extract and Verify the Hidden Evidence

Extract the concealed file using the correct password.

Example:

```
mkdir -p extracted
steghide extract -sf tower_stego_lab4.bmp -p 1234
```

Compare the original and extracted files using diff and/or SHA-256 hashes. The recovered file should match the original if extraction is successful.

```
(water@kali)-[~/Desktop]
$ steghide extract -sf _tower_original_image_for_lab.bmp -xf asecret.txt -p 1234
wrote extracted data to "asecret.txt".

(water@kali)-[~/Desktop]
$ ls
120M      coded_again.jpg      knock-env      reconstructed.jpg      usb_fat_carving.001
120M.7z   File_carving.docx    lab            secret.txt             usb_fat_carving.001.txt
asecret.txt      image_hex.txt      'lab file'     steg
asecret.txt      jpeg_strings.txt   NTFS.zip       _tower_original_image_for_lab.bmp
CIP-B101_Lab1C_wale_water  J_ub_law.jpg      output         usb

(water@kali)-[~/Desktop]
$ cat asecret.txt
Full Name: Olanrewaju Wale Mustapha Oladimeji
Evidence Message: This is a controlled evidence note created for the steganography practical lab.
Confirmation: This file was created for the ICDEA lab.

(water@kali)-[~/Desktop]
$

(water@kali)-[~/Desktop]
$ md5sum asecret.txt
6c1617d9a643e3e0a2cff65c458b21f3  asecret.txt

(water@kali)-[~/Desktop]
$
```

Part E — Compare Original and Stego Images

Compare:

- File sizes
- SHA-256 hashes
- Initial bytes using xxd
- Readable strings where applicable

Explain why two images may look visually similar while their underlying digital content and hashes are different.

- Initial bytes using xxd

```
(water@kali)-[~/Desktop]
└─$ xxd _tower_original_image_for_lab.bmp
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6.....6...(.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  ..... "5 #8. 8.
00000040: 203a 2425 3f28 2945 2729 482a 2b4d 2a2c  :$%?( )E' )H*+M*,
00000050: 4e29 2e4f 282e 5127 3052 2831 5328 3254  N).O(.Q'OR(1S(2T
00000060: 2f20 2b2f 282e 5127 3052 2831 5328 3254  N).O(.Q'OR(1S(2T
```

```
(water@kali)-[~/Desktop]
└─$ cd steg

(water@kali)-[~/Desktop/steg]
└─$ ls
_tower_original_image_for_lab.bmp

(water@kali)-[~/Desktop/steg]
└─$ xxd _tower_original_image_for_lab.bmp
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6.....6...(.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  ..... "5 #8. 8.
00000040: 203a 2425 3f28 2945 2729 482a 2b4d 2a2c  :$%?( )E' )H*+M*,
00000050: 4e29 2e4f 282e 5127 3052 2831 5328 3254  N).O(.Q'OR(1S(2T
```

- SHA-256 hashes

```
(water@kali)-[~/Desktop/steg]
└─$ ls
_tower_original_image_for_lab.bmp

(water@kali)-[~/Desktop/steg]
└─$ md5sum _tower_original_image_for_lab.bmp
7f77e02253d5ae6426c4d0c932325ae3 _tower_original_image_for_lab.bmp
```

```
(water@kali)-[~/Desktop]
└─$ md5sum _tower_original_image_for_lab.bmp
5a775dba746117c61d8cfc420a4d3 _tower_original_image_for_lab.bmp

(water@kali)-[~/Desktop]
└─$
```

- File sizes

```
(water@kali)-[~/Desktop/steg]
└─$ file _tower_original_image_for_lab.bmp
_tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54

(water@kali)-[~/Desktop/steg]
└─$
```

```
(water@kali)-[~/Desktop]
└─$ file _tower_original_image_for_lab.bmp
_tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54

(water@kali)-[~/Desktop]
└─$
```

- Explain why two images may look visually similar while their underlying digital content and hashes are different

The two images may look the same because the hidden information is added in very small parts of the image. These small changes are usually not visible to our eyes. However, the computer sees these changes as different data. Because the data is different, the SHA-256 hashes of the two images will also be different. Therefore, an image can look the same but still contain hidden information

Part F — Controlled Password Dictionary Test

Create a small lab wordlist that contains the known password 1234.

Use StegSeek against the stego image:

```
stegseek tower_stego_lab4.bmp lab4_wordlist.txt
```

Document the result.

If StegSeek is unavailable or fails in your environment, document the error clearly and complete the Steghide embedding/extraction workflow successfully instead.

```
(water@kali)-[~/Desktop]
└─$ ls /usr/share/wordlists/
amass  dirbuster  fasttrack.txt  john.lst  metasploit  rockyou.txt  wfuzz
dirb   dnsmap.txt  fern-wifi     legion    nmap.lst    sqlmap.txt   wifite.txt

(water@kali)-[~/Desktop]
└─$ ls
120M   coded_again.jpg  knock-env  reconstructed.jpg  usb_fat_carving.001
120M.7z  File_carving.docx  lab        secret.txt         usb_fat_carving.001.txt
asecret.txt  image_hex.txt  'lab file'  steg
asecret.txt  jpeg_strings.txt  NTFS.zip   _tower_original_image_for_lab.bmp
CIP-B101_Lab1C_wale_water  J_ub_law.jpg  output     usb

(water@kali)-[~/Desktop]
└─$ stegcracker _tower_original_image_for_lab.bmp /usr/share/wordlists/rockyou.txt
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2026 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek

Counting lines in wordlist..
Attacking file '_tower_original_image_for_lab.bmp' with wordlist '/usr/share/wordlists/rockyou.txt'..
```

```
(water@kali)-[~/Desktop]
└─$ stegcracker _tower_original_image_for_lab.bmp /usr/share/wordlists/rockyou.txt
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2026 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek

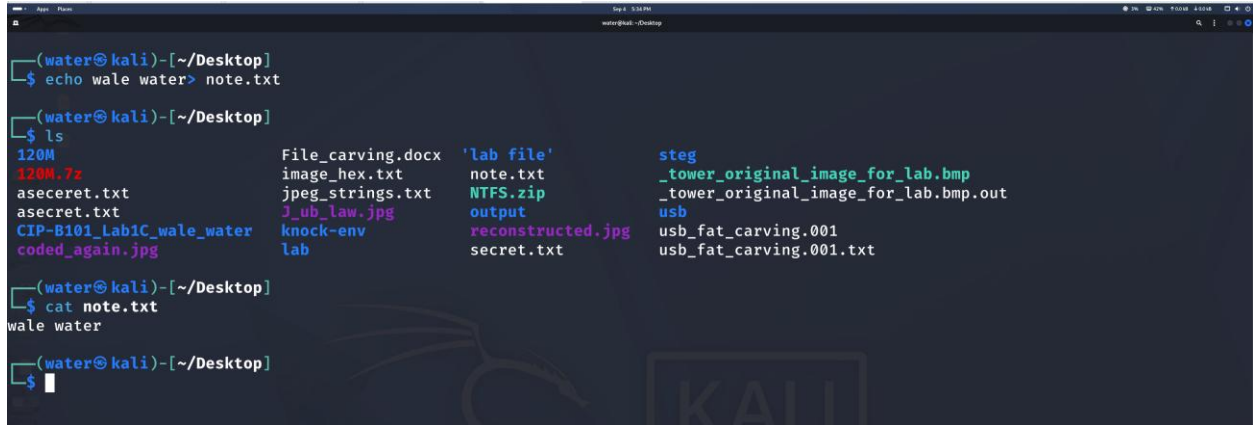
Counting lines in wordlist..
Attacking file '_tower_original_image_for_lab.bmp' with wordlist '/usr/share/wordlists/rockyou.txt'..
Successfully cracked file with password: 1234
Tried 2037 passwords
Your file has been written to: _tower_original_image_for_lab.bmp.out
1234

(water@kali)-[~/Desktop]
└─$
```


Part G — Independent Student Task

Create your own hidden evidence file:

firstname_lastname_hidden_note.txt



```
(water@kali)-[~/Desktop]
$ echo wale water > note.txt

(water@kali)-[~/Desktop]
$ ls
120M.7z          File_carving.docx  'lab file'      steg
120M.7z          image_hex.txt      note.txt         _tower_original_image_for_lab.bmp
asecret.txt      jpeg_strings.txt   NTFS.zip        _tower_original_image_for_lab.bmp.out
asecret.txt      J_ub_low.jpg       output          usb
CIP-B101_Lab1C_wale_water  knock-env         reconstructed.jpg  usb_fat_carving.001
coded_again.jpg  lab                secret.txt      usb_fat_carving.001.txt

(water@kali)-[~/Desktop]
$ cat note.txt
wale water

(water@kali)-[~/Desktop]
$
```

Write a short explanation of what steganography is and why it matters in digital forensics.

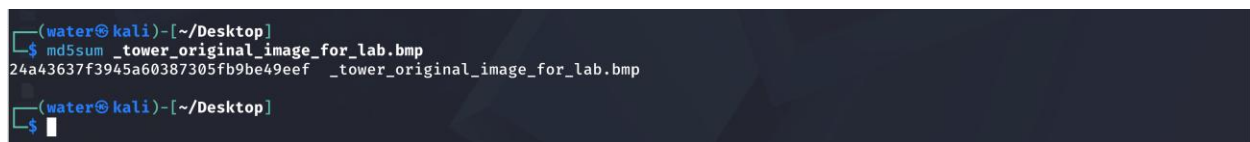
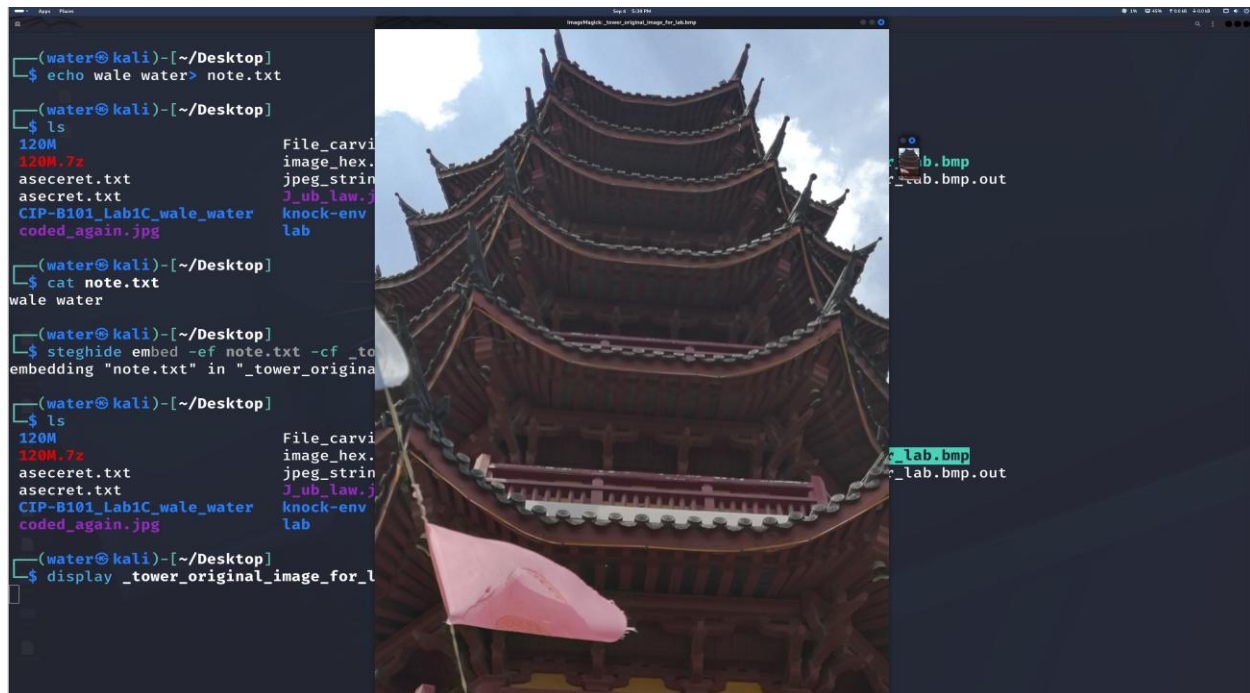
Steganography is a way of hiding a message or file inside another file, such as a picture, so that people do not easily notice it. It is important in digital forensics because someone may use it to hide important or suspicious information. A forensic investigator needs to know how to find and recover this hidden information. This can help reveal useful evidence during an investigation.

Then:

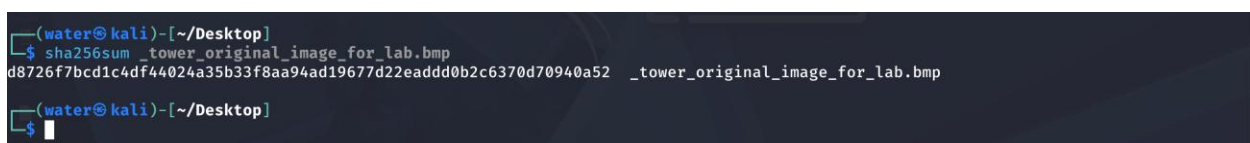
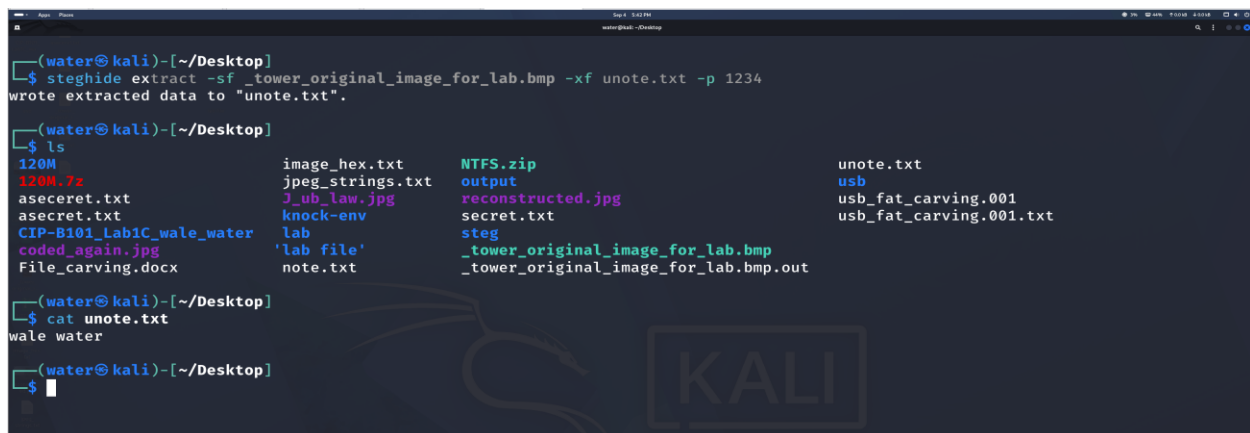
1. Embed it into the carrier image.
2. Use a password of your choice.
3. Extract it again.
4. Verify integrity using hashes.
5. Create a small wordlist containing your chosen password.
6. Test the recovery workflow.
7. Document every step with screenshots.

1.Embed it into the carrier image.

2.Use a password of your choice.



3.Extract it again.



2.7. Analysis and Findings

The practical helped me understand how a file can be hidden inside an image. I used Steghide to hide and recover the secret file with a password. I used SHA 256 to check that the recovered file was the same as the original file. I also learned that two images can look the same but have different file information and hashes. The practical showed me how hidden information can be useful when looking for digital evidence.

2.8. Challenges and Solutions

One challenge was understanding the commands used to hide and extract the file. I also had to make sure I used the correct password when recovering the hidden file. I solved these problems by carefully following the lab instructions and checking my commands. I also used hashes to make sure the recovered file was correct. The practical became easier after practising the commands.

2.9. Conclusion

This practical helped me understand how steganography works. I learned how to hide a file inside an image and recover it again using Steghide. I also learned how to use SHA-256 to check if a file has been changed. The practical showed me that hidden information may not be visible by simply looking at an image. Overall, I gained useful knowledge about finding and checking hidden digital evidence.

2.10. Recommendations

Regular Assignment like this will help students improve their knowledge

2.11. References

Class Video Guide from Founder ICDFIA on Computer and Digital Forensic

Digital Forensics Basics: A Step-by-Step Guide for Beginners — Companion Materials guide from [GitHub - frankwxu/Digital-Forensics-Basic-Book · GitHub](#)